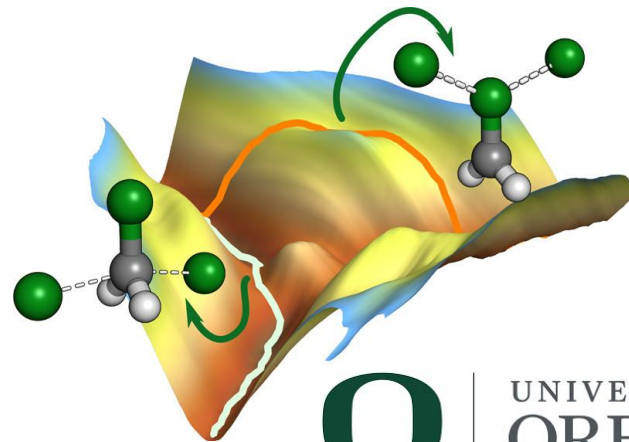
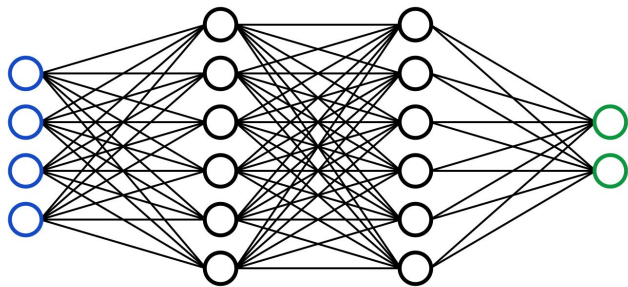


Machine Learning for Chemistry

Winter 2025

Topic 6: Dimensionality Reduction



Instructor: Dr. Dhiman Ray

Assistant Professor, Department of Chemistry and Biochemistry

Overview of Topics

- Principal Component Analysis (PCA)
- Time-lagged Independent Component Analysis (TICA)
- Linear Discriminant Analysis (LDA)
- Non-linear dimensionality reduction (Kernel-PCA and t-SNE)

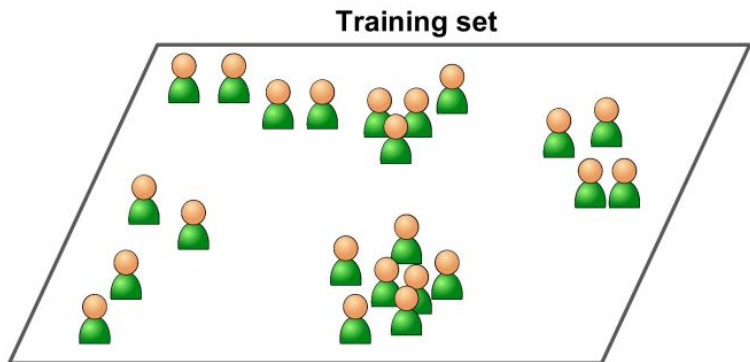
References:

1. **Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow** by Aurélien Géron (Chapter 8)
2. **Machine Learning with Pytorch and Scikit-Learn** by Raschka, Liu, and Mirjalili (Chapter 5)

Unsupervised Learning

Finding patterns in unlabelled training data.

1. Dimensionality Reduction (this lesson)
2. Clustering (Done Earlier)



Supervised learning

- > Labeled data
- > Direct feedback
- > Predict outcome/future

Unsupervised learning

- > No labels/targets
- > No feedback
- > Find hidden structure in data

Reinforcement learning

- > Decision process
- > Reward system
- > Learn series of actions

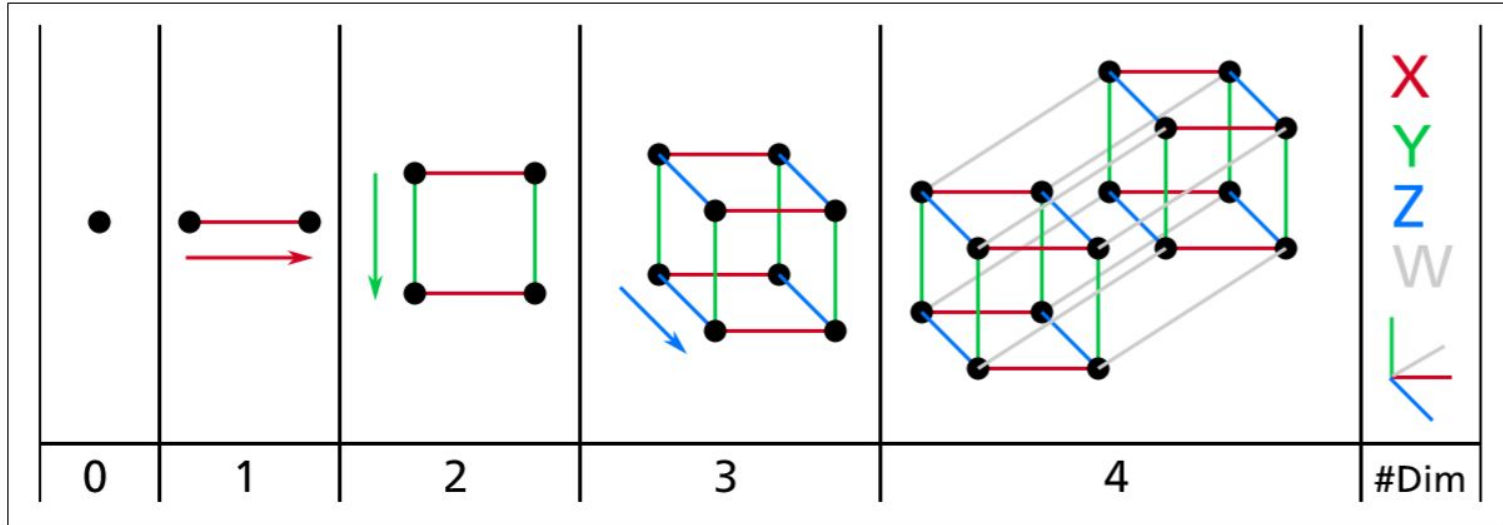
Dimensionality Reduction

But why are high dimensions problematic?

1. Visualization
 2. Sparsity
 3. Distance Measurement
- etc.

Curse of Dimensionality: Visualization

Visualization of more than 3 dimension is hard



In Machine Learning **each feature corresponds to one dimension** leading to very high dimensional spaces

Curse of Dimensionality: Sparsity

If you **pick two points randomly** in a **unit square**, the distance between these two points will be, on average, roughly 0.52. If you pick two random points in a unit **3D cube**, the average distance will be roughly 0.66.

But what about two points picked randomly in a **1,000,000-dimensional hypercube**? Well, the average distance, will be about 408.25

- High dimensional datasets are at risk of being very sparse. **Any new instance will be far away from any training instance**, making predictions much less reliable than in lower dimensions.
- The amount of training data needed to fit a model grows exponentially with increasing dimension.

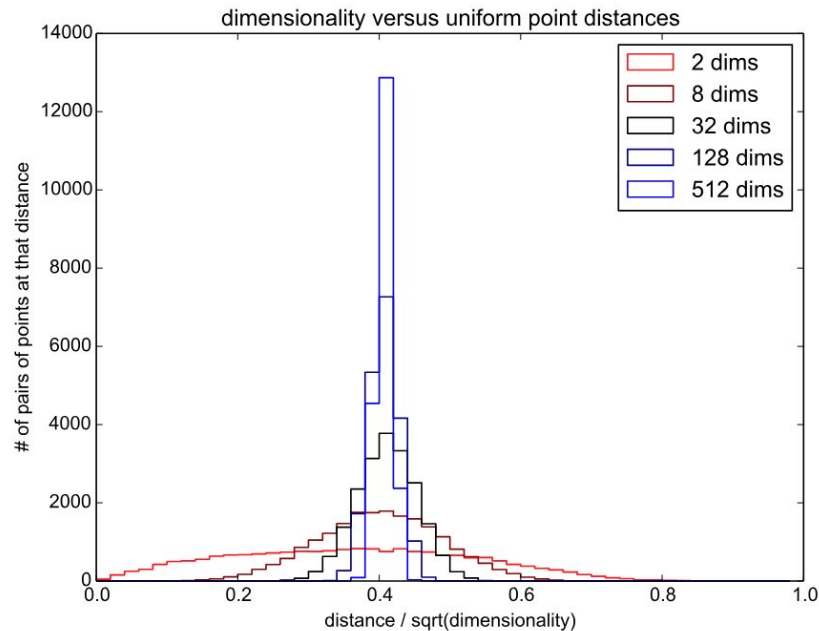
Curse of Dimensionality: Distance Measurement

Effective variance of the distance between points shrinks with increasing dimension.

Most of ML algorithms critically depends on accurately measuring distances between points (e.g. Loss functions)

In higher dimensions, the concept of two points being near of far does not make sense.

See “A course in Machine Learning” by Daume. Section 3.5 for a rigorous explanation.



Dimensionality Reduction

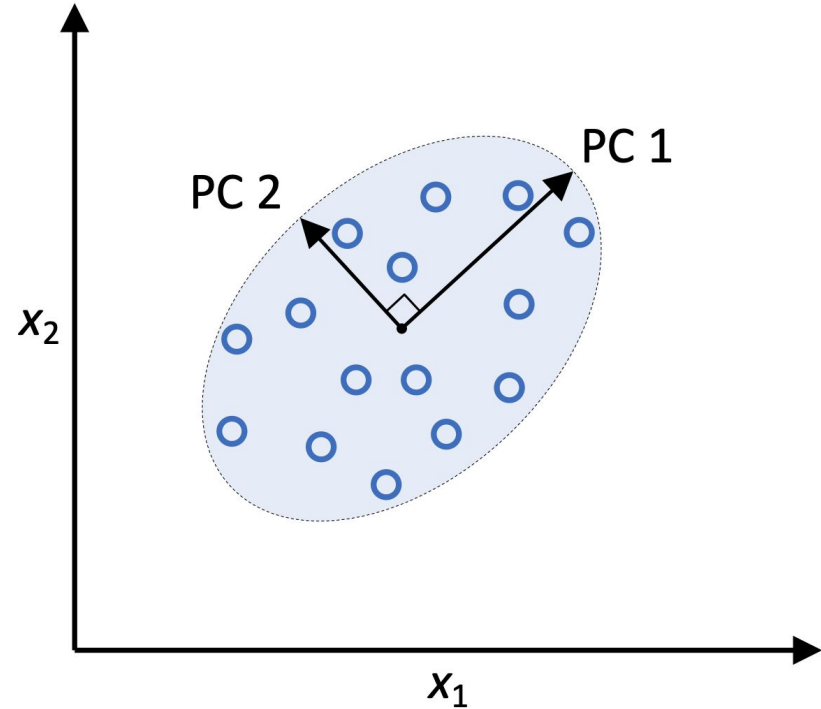
Summarizing the information content of a dataset by transforming it onto a subspace of lower dimensionality.

Data compression is an important topic in machine learning, and it helps us to store and analyze the increasing amounts of data that are produced and collected in the modern age of technology

Principal Component Analysis

Principal Component Analysis (PCA) helps us to identify patterns in data based on the correlation between features.

PCA aims to find the directions of maximum variance in high dimensional data and projects the data onto a new subspace with equal or fewer dimensions than the original one. The orthogonal axes (principal components) of the new subspace can be interpreted as the directions of maximum variance.



Principal Component Analysis

If we use PCA for dimensionality reduction, we construct a $d \times k$ -dimensional transformation matrix, W , that allows us to map a vector of the features of the training example, $\mathbf{x}$, onto a new k -dimensional feature subspace that has fewer dimensions than the original d -dimensional feature space. For instance, the process is as follows. Suppose we have a feature vector, $\mathbf{x}$:

$$\mathbf{x} = [x_1, x_2, \dots, x_d], \quad \mathbf{x} \in \mathbb{R}^d$$

which is then transformed by a transformation matrix, $W \in \mathbb{R}^{d \times k}$:

$$\mathbf{x}W = \mathbf{z}$$

resulting in the output vector:

$$\mathbf{z} = [z_1, z_2, \dots, z_k], \quad \mathbf{z} \in \mathbb{R}^k$$

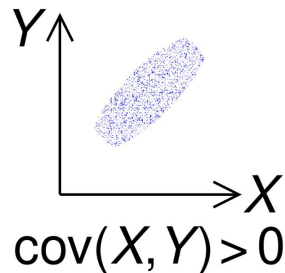
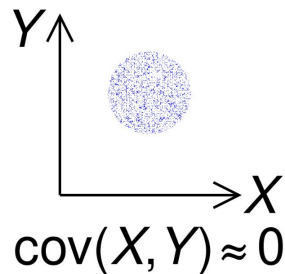
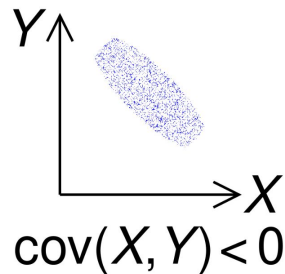
Covariance Matrix

The covariance between two features, x_j and x_k , can be calculated via the following equation:

$$\sigma_{jk} = \frac{1}{n-1} \sum_{i=1}^n (x_j^{(i)} - \mu_j) (x_k^{(i)} - \mu_k)$$

The symmetric $d \times d$ -dimensional covariance matrix (Σ), where d is the number of dimensions in the dataset, stores the pairwise covariances between the different features.

$$\Sigma = \begin{bmatrix} \sigma_1^2 & \sigma_{12} & \sigma_{13} \\ \sigma_{21} & \sigma_2^2 & \sigma_{23} \\ \sigma_{31} & \sigma_{32} & \sigma_3^2 \end{bmatrix}$$



Eigendecomposition of Covariance Matrix

The eigenvectors $\mathbf{v}$ and the corresponding eigenvalues λ of the covariance matrix (Σ) are computed as:

$$\Sigma \mathbf{v} = \lambda \mathbf{v}$$

There exists d eigenvectors and each eigenvector has d dimensions.

The first k eigenvectors correspond to the first k principal components.

Therefore the $d \times k$ -dimensional transformation matrix ($\mathbf{W}$) is constructed by concatenating the first k eigenvectors. $\mathbf{W} = (\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k)$

$$\mathbf{x}\mathbf{W} = \mathbf{z}$$

$$\mathbf{z} = [z_1, z_2, \dots, z_k], \quad \mathbf{z} \in \mathbb{R}^k$$

Steps in the PCA Algorithm

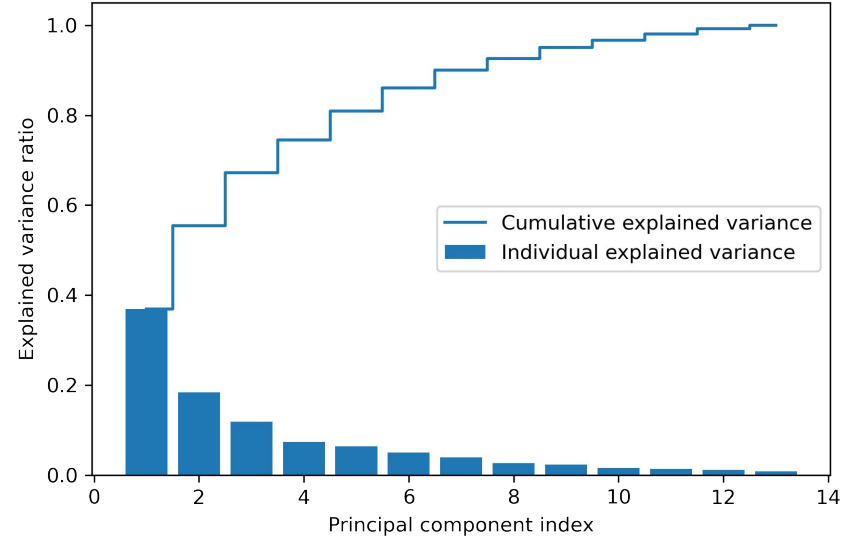
1. Standardize the d -dimensional dataset.
2. Construct the covariance matrix.
3. Decompose the covariance matrix into its eigenvectors and eigenvalues.
4. Sort the eigenvalues by decreasing order to rank the corresponding eigenvectors.
5. Select k eigenvectors, which correspond to the k largest eigenvalues, where k is the dimensionality of the new feature subspace ($k \leq d$).
6. Construct a projection matrix, W , from the “top” k eigenvectors.
7. Transform the d -dimensional input dataset, X , using the projection matrix, W , to obtain the new k -dimensional feature subspace.

How many Principal Components should I use?

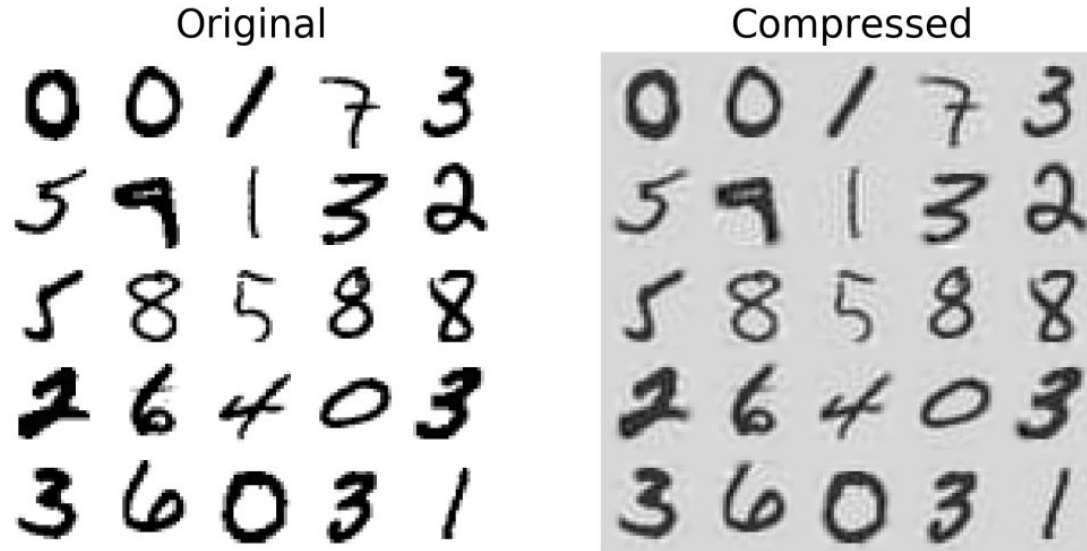
Explained Variance Ratio: The variance explained ratio of an eigenvalue, λ_j , is the fraction of an eigenvalue, λ_j , and the total sum of the eigenvalues:

$$\text{Explained variance ratio} = \frac{\lambda_j}{\sum_{j=1}^d \lambda_j}$$

Usually one chooses first k eigenvectors with the highest eigenvalues to keep a 95% cumulative explained variance.



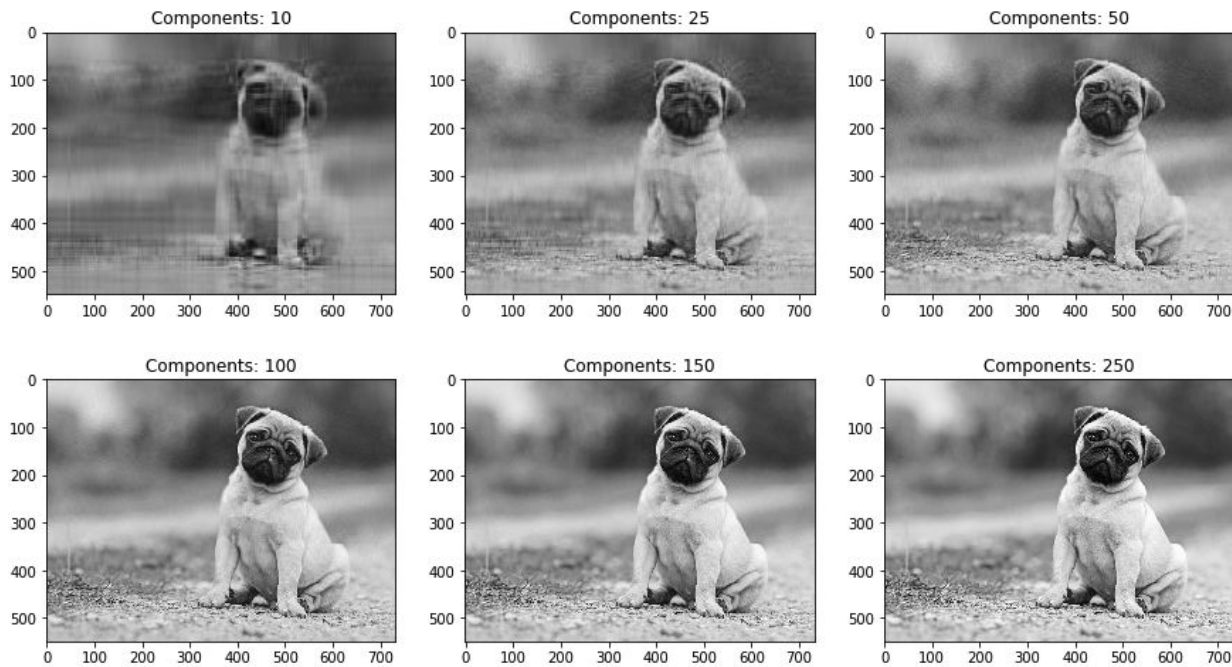
How many Principal Components should I use?



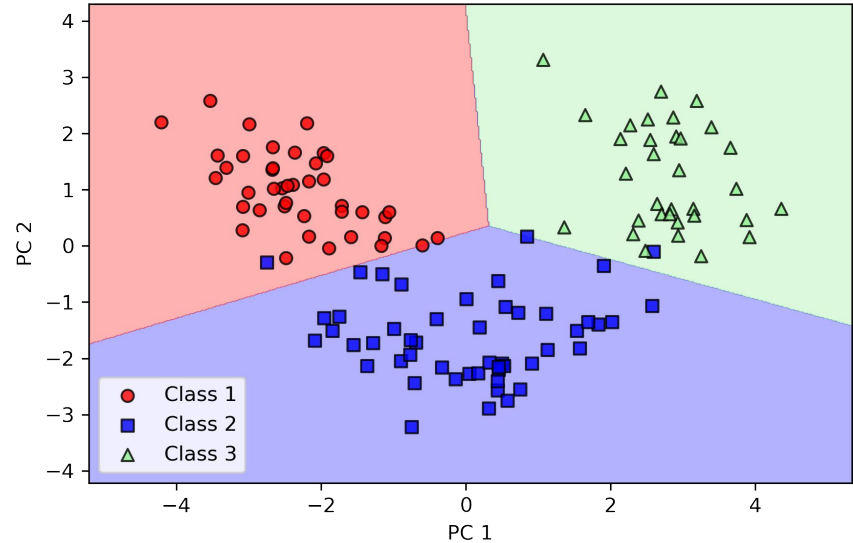
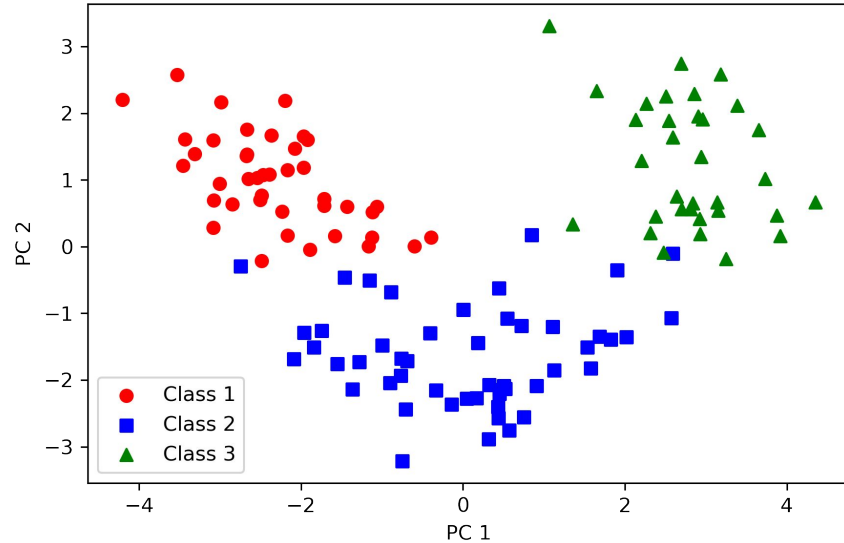
In the MNIST handwriting dataset each image is described by $28 \times 28 = 784$ pixels (features). PCA with 95% cumulative explained variance can compress the feature dimension to 154.

Reconstruction of original inputs from PCA

We can reconstruct the original inputs (x) by performing an inverse projection of the reduced dimensional representation (z): $x = zW^{-1}$



Use the reduced dimensional space for Machine Learning



Once the dataset has been projected onto the low dimensional space we can train Machine Learning models (e.g. Clustering, Classification, Regression, etc.) directly in the reduced dimensional space without touching the high dimensional data.

Application of PCA: Analysis of Protein Folding Simulation

Protein Structure

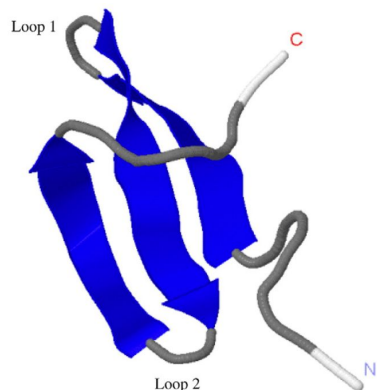
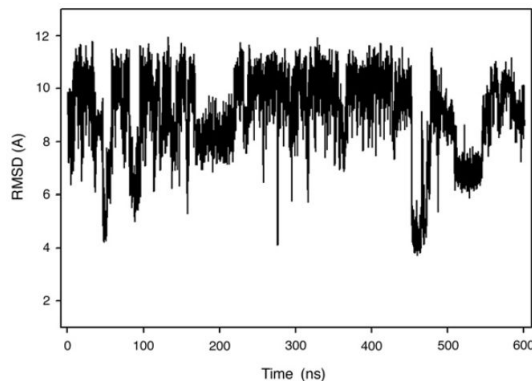
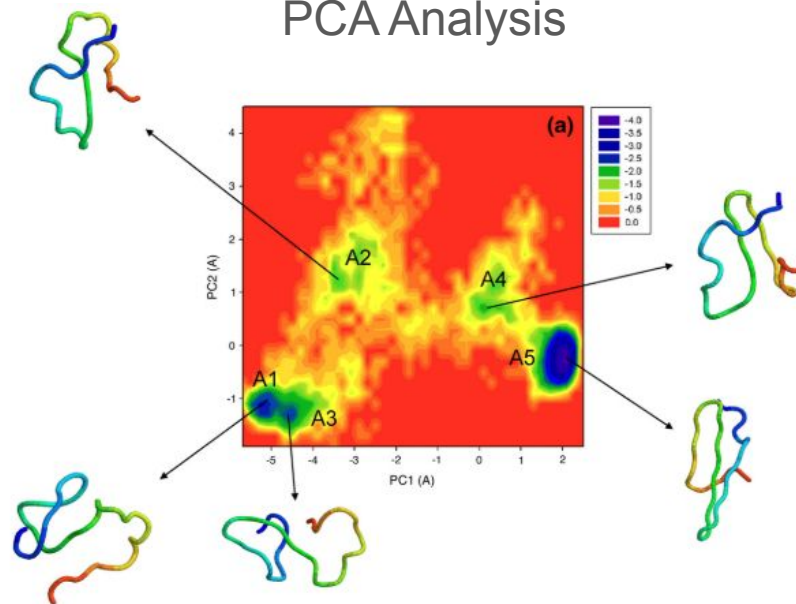


Fig. 1. Experimental NMR structure¹ of the triple β -strand WW domain from FBP28 (1E0L).

Molecular Dynamics Simulation



PCA Analysis

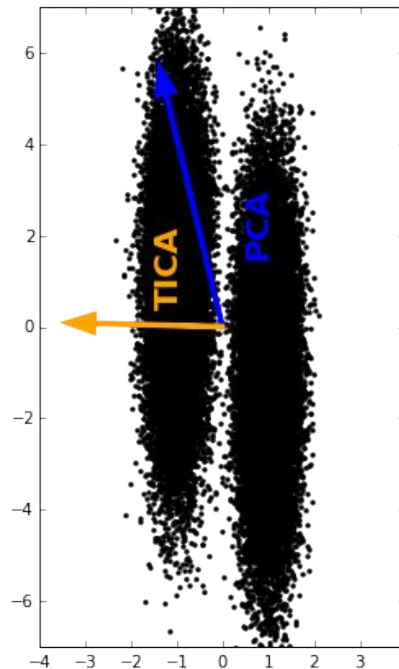


- Maisuradze, Gia G., Adam Liwo, and Harold A. Scheraga. "Principal component analysis for protein folding dynamics." *Journal of molecular biology* 385, no. 1 (2009): 312-329.
- Ernst, Matthias, Florian Sittel, and Gerhard Stock. "Contact-and distance-based principal component analysis of protein dynamics." *The Journal of chemical physics* 143, no. 24 (2015).

Time-lagged Independent Component Analysis (TICA)

In time-series analysis such as molecular dynamics simulation we often need to find out the slowest degree of freedom rather than the one with highest variance.

Time-lagged Independent Component Analysis (TICA) solves this issue by performing eigendecomposition of the time-lagged covariance matrix instead of the original covariance matrix.



Time-lagged Independent Component Analysis (TICA)

First, standardize the data by removing the mean:

$$\mathbf{r}(t) = \tilde{\mathbf{r}}(t) - \langle \tilde{\mathbf{r}}(t) \rangle_t$$

Construct the time lagged covariance matrix $\mathbf{C}(\tau)$ with lag time τ :

$$c_{ij}(\tau) = \langle r_i(t) r_j(t + \tau) \rangle_t = \frac{1}{N - \tau - 1} \sum_{t=1}^{N-\tau} r_i(t) r_j(t + \tau)$$

The matrix $\mathbf{C}(0)$ is the normal covariance matrix as there is not time lag.

$$\sigma_{jk} = \frac{1}{n-1} \sum_{i=1}^n (x_j^{(i)} - \mu_j) (x_k^{(i)} - \mu_k)$$

Time-lagged Independent Component Analysis (TICA)

Now we solve the generalized eigenvalue problem:

$$\mathbf{C}(\tau) \mathbf{U} = \mathbf{C}(0) \mathbf{U} \mathbf{\Lambda}$$

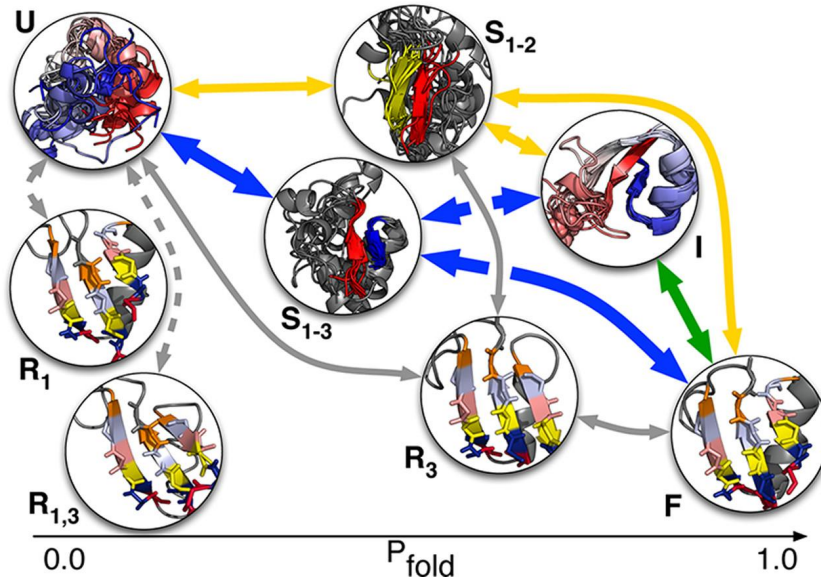
where $\mathbf{U}$ is the eigenvector-matrix containing the independent components (ICs) in the columns and $\mathbf{\Lambda}$ is a diagonal eigenvalue matrix. This problem can be solved by an appropriate generalized eigenvalue solver.

Now, the data can be projected onto the TICA space:

$$\mathbf{z}^\top(t) = \mathbf{r}^\top(t) \mathbf{U}$$

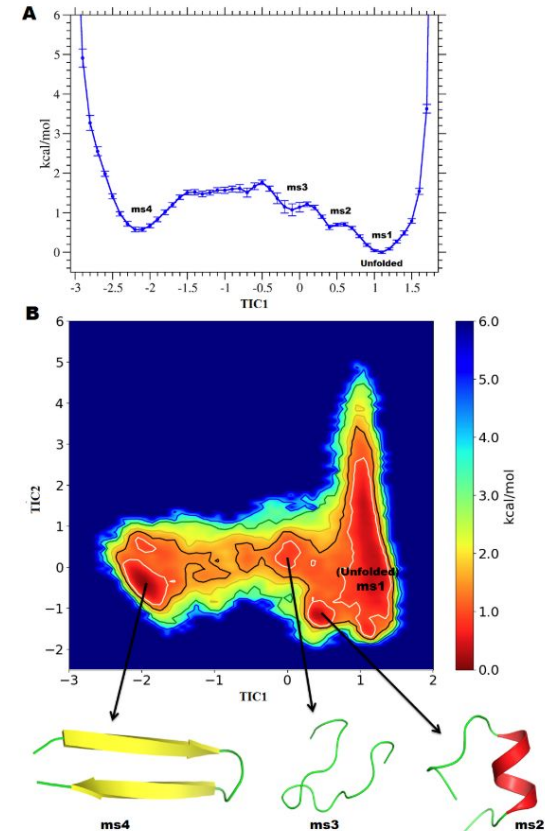
In this step, we can perform a dimension reduction by selecting only a sub-matrix $\mathbf{U}$ consisting of the first m columns of the full-rank $\mathbf{U}$

Application of TICA: protein folding simulation



Schwantes, Christian R., and Vijay S. Pande. "Improvements in Markov state model construction reveal many non-native interactions in the folding of NTL9." *Journal of chemical theory and computation* 9, no. 4 (2013): 2000-2009.

Ahalawat, Navjeet, and Jagannath Mondal. "Assessment and optimization of collective variables for protein conformational landscape: GB1 β -hairpin as a case study." *The Journal of chemical physics* 149, no. 9 (2018).



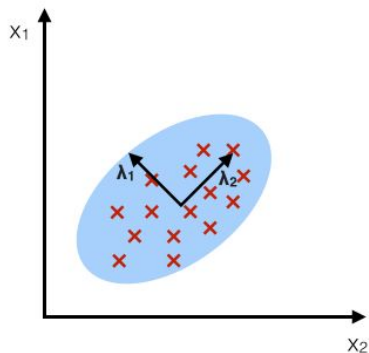
Jupyter Notebook Tutorial

Linear Discriminant Analysis (LDA)

LDA is a **supervised learning** algorithm to find a low-dimensional feature subspace that maximises the separation between two or more classes.

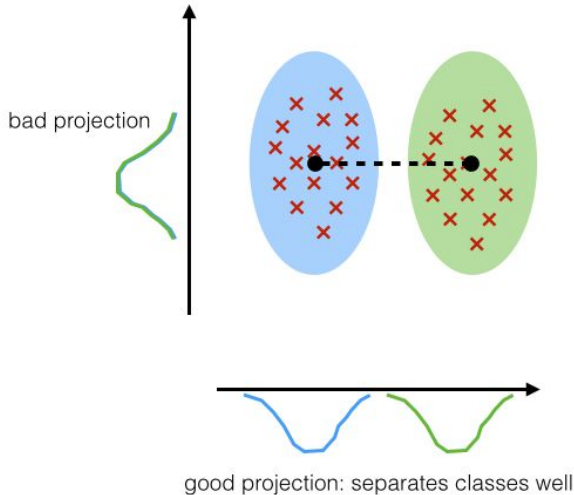
PCA:

component axes that maximize the variance



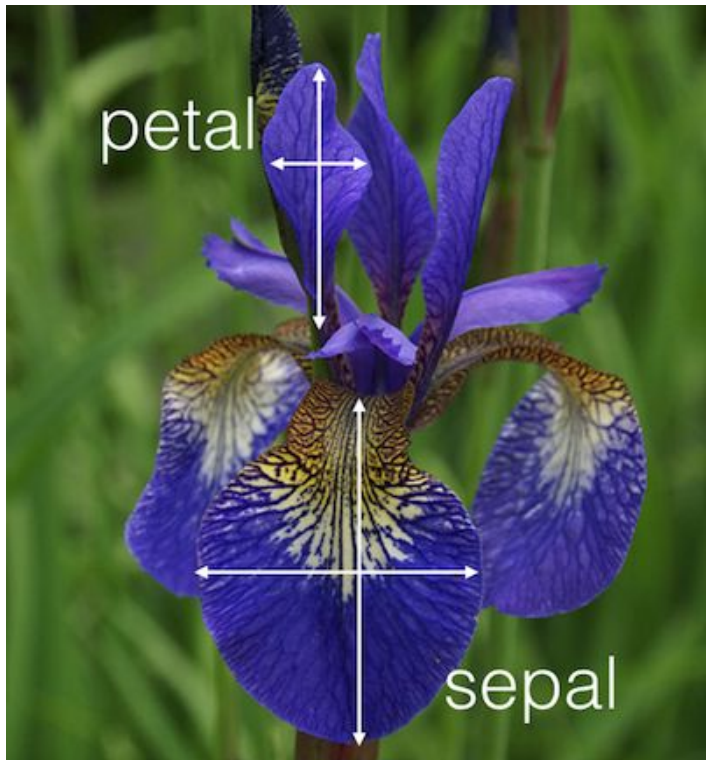
LDA:

maximizing the component axes for class-separation



Linear Discriminant Analysis (LDA)

We will go through the LDA algorithm step-by-step for the iris dataset.

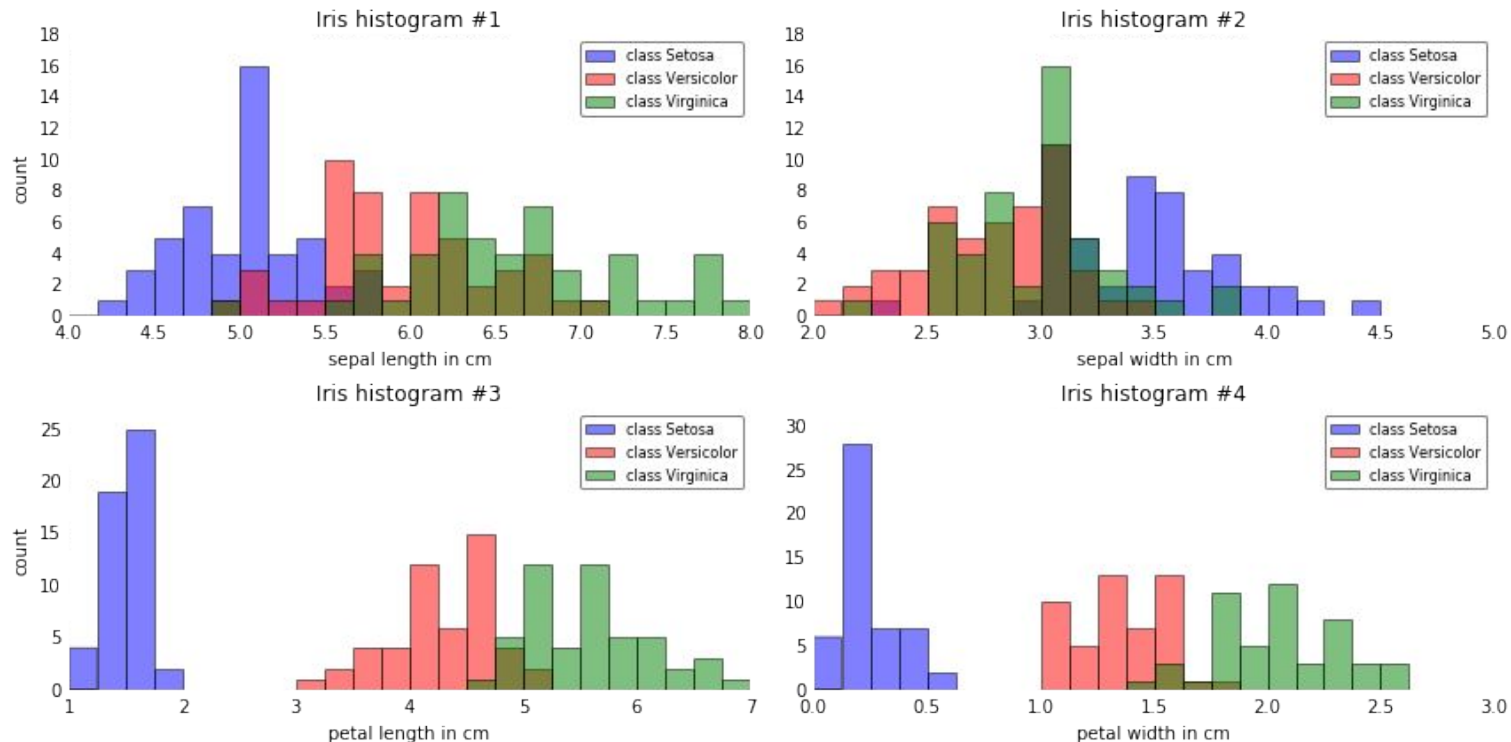


	sepal length in cm	sepal width in cm	petal length in cm	petal width in cm	class label
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

$$\mathbf{X} = \begin{bmatrix} x_{1_{\text{sepal length}}} & x_{1_{\text{sepal width}}} & x_{1_{\text{petal length}}} & x_{1_{\text{petal width}}} \\ x_{2_{\text{sepal length}}} & x_{2_{\text{sepal width}}} & x_{2_{\text{petal length}}} & x_{2_{\text{petal width}}} \\ \dots & & & \\ x_{150_{\text{sepal length}}} & x_{150_{\text{sepal width}}} & x_{150_{\text{petal length}}} & x_{150_{\text{petal width}}} \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} \omega_{\text{setosa}} \\ \omega_{\text{setosa}} \\ \dots \\ \omega_{\text{virginica}} \end{bmatrix}$$

Linear Discriminant Analysis (LDA)

Some of the features discriminate the species very well while some don't.



Linear Discriminant Analysis (LDA) in 5 steps

Step 1: Computing the d-dimensional mean vectors

In this first step, we will start off with a simple computation of the mean vectors $\mathbf{m}_i$, ($i = 1, 2, 3$) of the 3 different flower classes:

$$\mathbf{m}_i = \begin{bmatrix} \mu_{\omega_i}(\text{sepal length}) \\ \mu_{\omega_i}(\text{sepal width}) \\ \mu_{\omega_i}(\text{petal length}) \\ \mu_{\omega_i}(\text{petal width}) \end{bmatrix}, \quad \text{with } i = 1, 2, 3$$

Linear Discriminant Analysis (LDA) in 5 steps

Step 2: Computing the Scatter Matrices

Now, we will compute the two 4×4 -dimensional matrices: The within-class and the between-class scatter matrix.

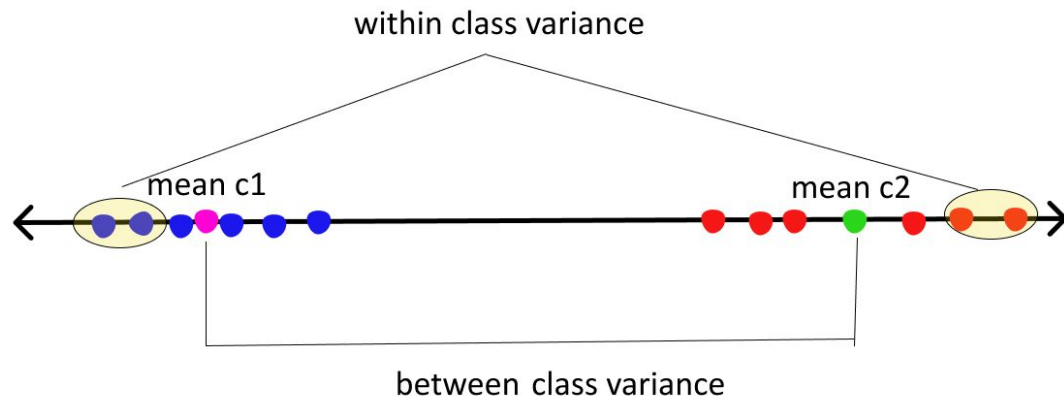


Figure Courtesy: Dash S.K.
A brief introduction to Linear
Discriminant Analysis

Linear Discriminant Analysis (LDA) in 5 steps

2.1 Within-class scatter matrix S_W

The **within-class scatter** matrix S_W is computed by the following equation:

$$S_W = \sum_{i=1}^c S_i$$

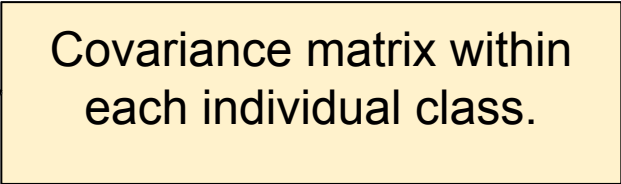
where

$$S_i = \sum_{\mathbf{x} \in D_i}^n (\mathbf{x} - \mathbf{m}_i) (\mathbf{x} - \mathbf{m}_i)^T$$

(scatter matrix for every class)

and $\mathbf{m}_i$ is the mean vector

$$\mathbf{m}_i = \frac{1}{n_i} \sum_{\mathbf{x} \in D_i}^n \mathbf{x}_k$$



Covariance matrix within
each individual class.

Linear Discriminant Analysis (LDA) in 5 steps

2.2 Between-class scatter matrix S_B

The **between-class scatter** matrix S_B is computed by the following equation:

$$S_B = \sum_{i=1}^c N_i (\mathbf{m}_i - \mathbf{m})(\mathbf{m}_i - \mathbf{m})^T$$

where

$\mathbf{m}$ is the overall mean, and $\mathbf{m}_i$ and N_i are the sample mean and sizes of the respective classes.

Covariance matrix of the
mean values of all classes

Linear Discriminant Analysis (LDA) in 5 steps

Step 3: Solving the generalized eigenvalue problem for the matrix $S_W^{-1} S_B$

Next, we will solve the generalized eigenvalue problem for the matrix $S_W^{-1} S_B$ to obtain the linear discriminants.

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$$

where

$$\mathbf{A} = S_W^{-1} S_B$$

$$\mathbf{v} = \text{Eigenvector}$$

$$\lambda = \text{Eigenvalue}$$

Linear Discriminant Analysis (LDA) in 5 steps

Step 4: Selecting linear discriminants for the new feature subspace

4.1. Sorting the eigenvectors by decreasing eigenvalues

Eigenvalues in decreasing order:

32.2719577997

0.27756686384

5.71450476746e-15

5.71450476746e-15

Variance explained:

eigenvalue 1: 99.15%

eigenvalue 2: 0.85%

eigenvalue 3: 0.00%

eigenvalue 4: 0.00%

Linear Discriminant Analysis (LDA) in 5 steps

Note

If we take a look at the eigenvalues, we can already see that 2 eigenvalues are close to 0. The reason why these are close to 0 is not that they are not informative but it's due to floating-point imprecision. In fact, these two last eigenvalues should be exactly zero: In LDA, the number of linear discriminants is at most $c - 1$ where c is the number of class labels, since the in-between scatter matrix S_B is the sum of c matrices with rank 1 or less.

Linear Discriminant Analysis (LDA) in 5 steps

4.2. Choosing k eigenvectors with the largest eigenvalues

After sorting the eigenpairs by decreasing eigenvalues, it is now time to construct our $d \times k$ -dimensional eigenvector matrix $\mathbf{W}$ (here 4×2 : based on the 2 most informative eigenpairs) and thereby reducing the initial 4-dimensional feature space into a 2-dimensional feature subspace.

Linear Discriminant Analysis (LDA) in 5 steps

Step 5: Transforming the samples onto the new subspace

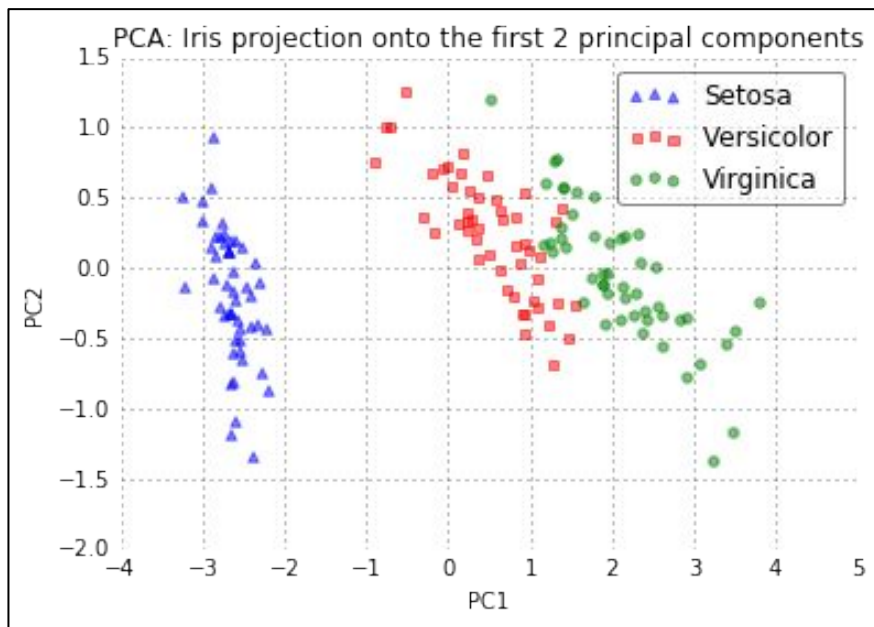
In the last step, we use the 4×2 -dimensional matrix $\mathbf{W}$ that we just computed to transform our samples onto the new subspace via the equation

$$\mathbf{Y} = \mathbf{X} \times \mathbf{W}.$$

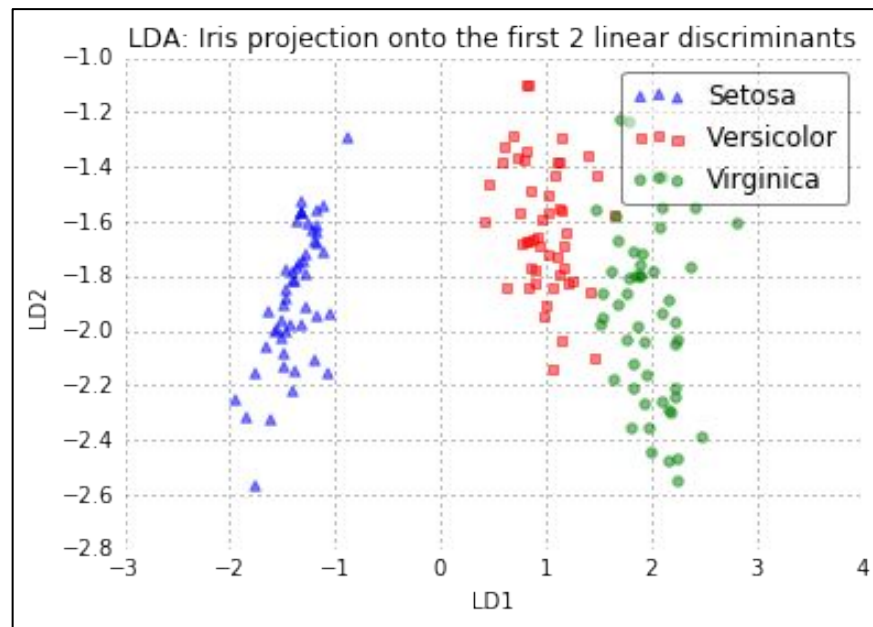
(where $\mathbf{X}$ is a $n \times d$ -dimensional matrix representing the n samples, and $\mathbf{Y}$ are the transformed $n \times k$ -dimensional samples in the new subspace).

Linear Discriminant Analysis (LDA)

PCA



LDA

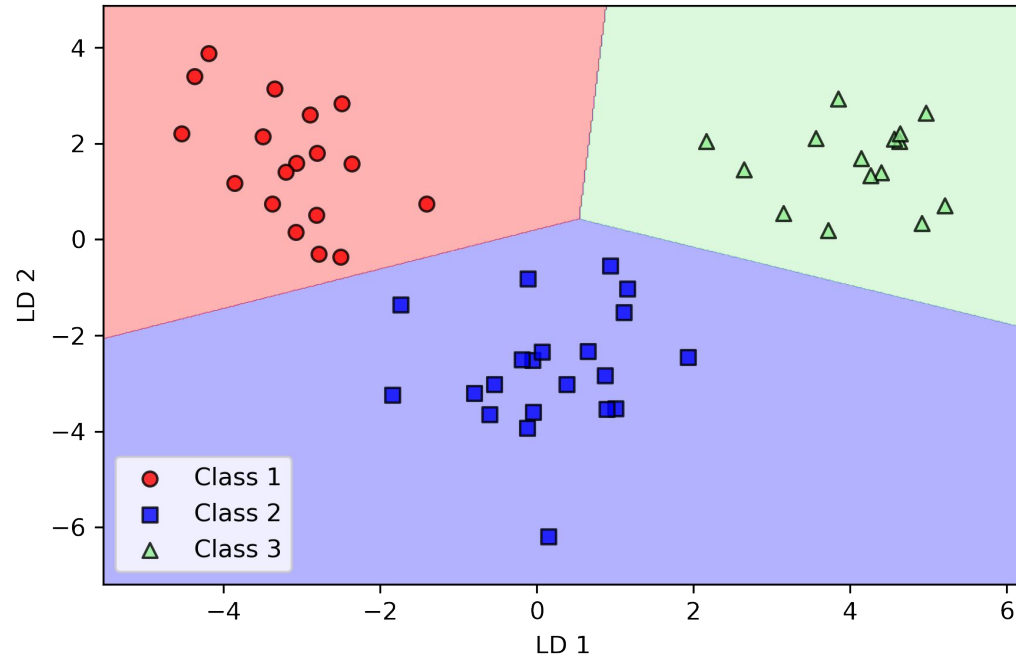


Summary of the LDA Algorithm

1. Compute the d -dimensional mean vectors for the different classes from the dataset.
2. Compute the scatter matrices (in-between-class and within-class scatter matrix).
3. Compute the eigenvectors ($\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_d$) and corresponding eigenvalues ($\lambda_1, \lambda_2, \dots, \lambda_d$) for the scatter matrices.
4. Sort the eigenvectors by decreasing eigenvalues and choose k eigenvectors with the largest eigenvalues to form a $d \times k$ dimensional matrix $\mathbf{W}$ (where every column represents an eigenvector).
5. Use this $d \times k$ eigenvector matrix to transform the samples onto the new subspace. This can be summarized by the matrix multiplication: $\mathbf{Y} = \mathbf{X} \times \mathbf{W}$ (where $\mathbf{X}$ is a $n \times d$ -dimensional matrix representing the n samples, and $\mathbf{y}$ are the transformed $n \times k$ -dimensional samples in the new subspace).

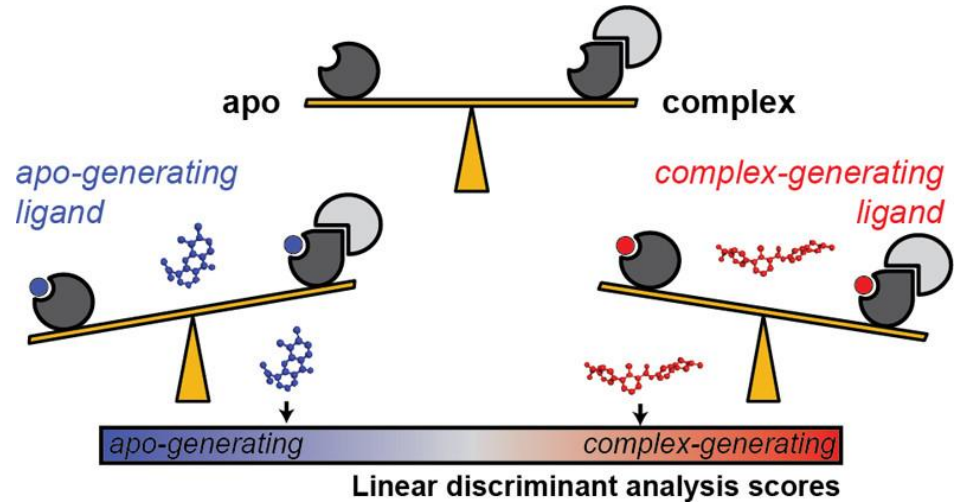
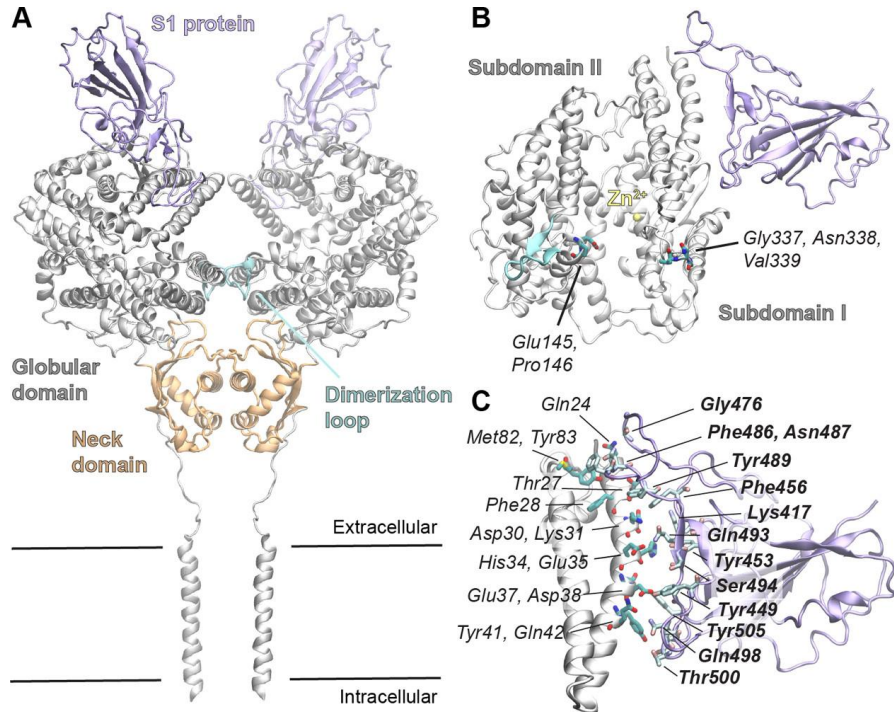
Using LDA coordinates as features

Similar to PCA, low dimensional LDA projections can be used as feature to perform machine learning tasks like regression or classification.



Application of LDA

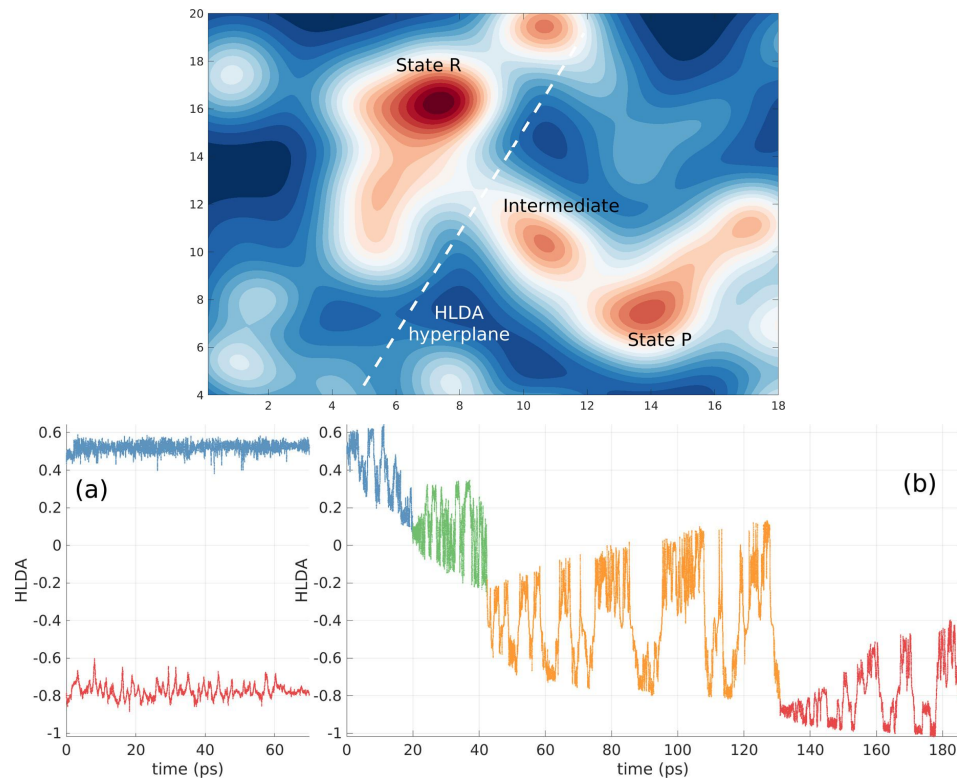
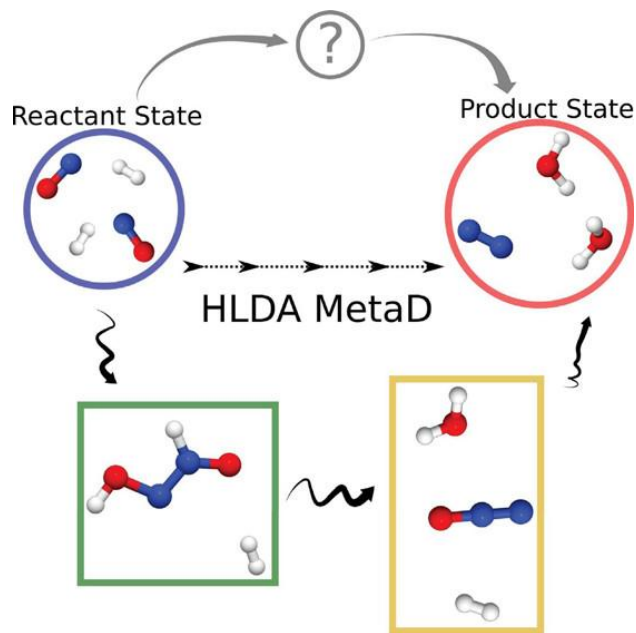
Classification of drugs that facilitate or inhibit the binding of Coronavirus spike protein to human ACE2 receptor.



Uyar, Arzu, and Alex Dickson. "Perturbation of ACE2 structural ensembles by SARS-CoV-2 spike protein binding." *Journal of Chemical Theory and Computation* 17, no. 9 (2021): 5896-5906.

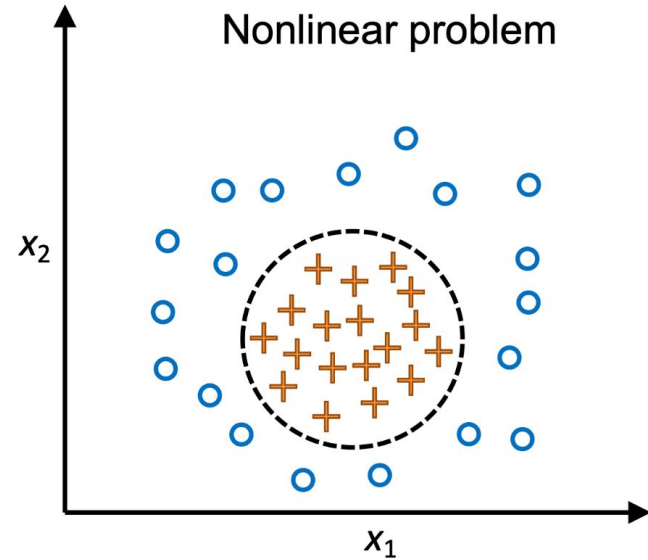
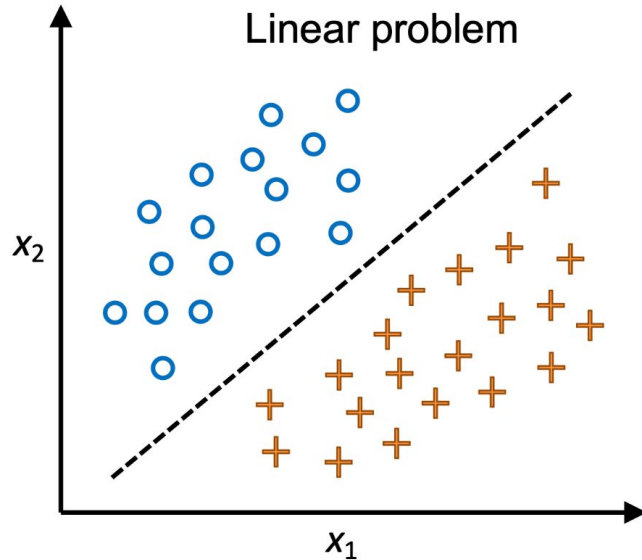
Application of LDA

Rizzi, Valerio, Dan Mendels, Emilia Sicilia, and Michele Parrinello. "Blind search for complex chemical pathways using harmonic linear discriminant analysis." *Journal of chemical theory and computation* 15, no. 8 (2019): 4507-4515.



Non-linear Dimensionality Reduction

If we are dealing with nonlinear problems, which we may encounter rather frequently in real-world applications, linear transformation techniques for dimensionality reduction, such as PCA and LDA, may not be the best choice

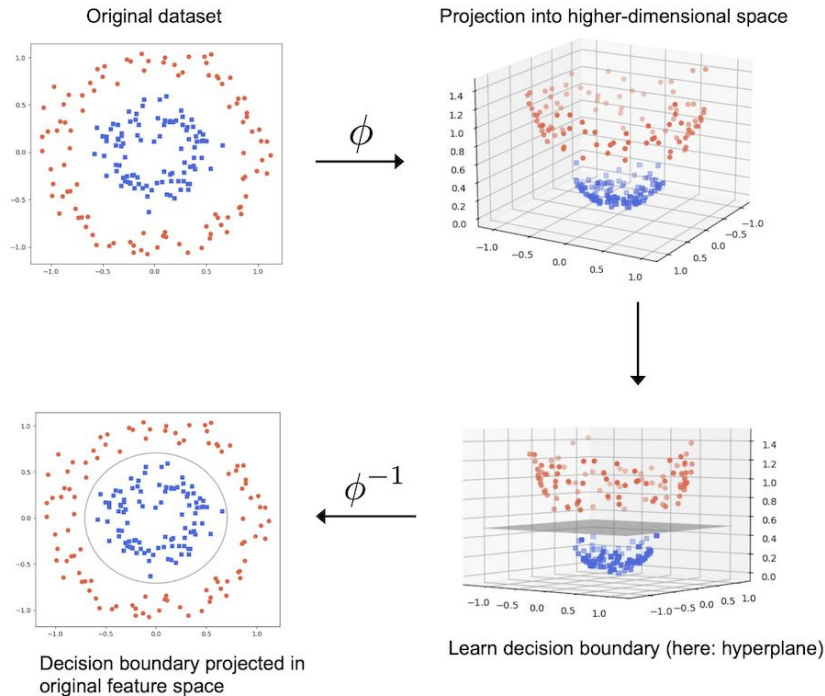


Kernel PCA

While N points cannot, in general, be linearly separated in $d < N$ dimensions, they can almost always be linearly separated in $d \geq N$ dimensions.

Project the N , d -dimensional data points into an N dimensional space. As we usually have more data points than features (d), this will extend the dimensionality.

Perform PCA in this high (N) dimensional space.



Kernel PCA

To avoid expensive matrix multiplications in the N dimensional space, us the kernel trick. We will construct an $N \times N$ dimensional Gram matrix (G) whose elements are given by:

$$G_{i,j} = K(x^{(i)}, x^{(j)}) = \exp \left(- \frac{\|x^{(i)} - x^{(j)}\|^2}{\sigma} \right)$$

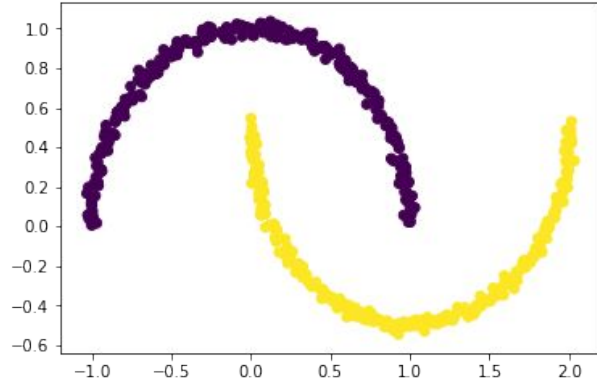
Where $x^{(i)}$ is the i -th training data and K is a gaussian kernel of width σ .

Now, the eigendecomposition of matrix G : $\mathbf{G}\alpha = \lambda N\alpha$ gives eigenvectors α and eigenvalues $N\lambda$. The eigenvectors have dimensions equal to number of datapoints.

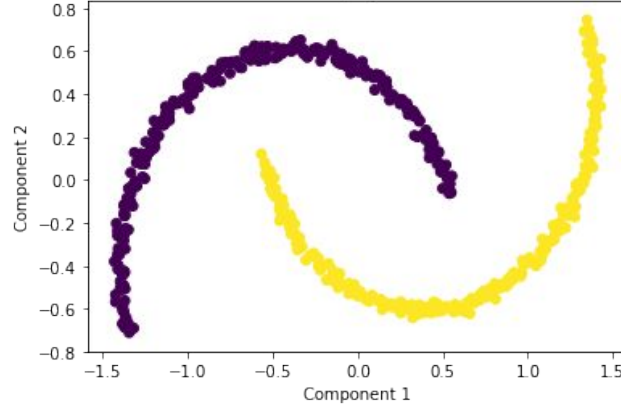
Projection of a new data point x along eigenvector $\alpha = \sum_{i=1}^N \alpha_i K(x, x^{(i)})$

Kernel PCA

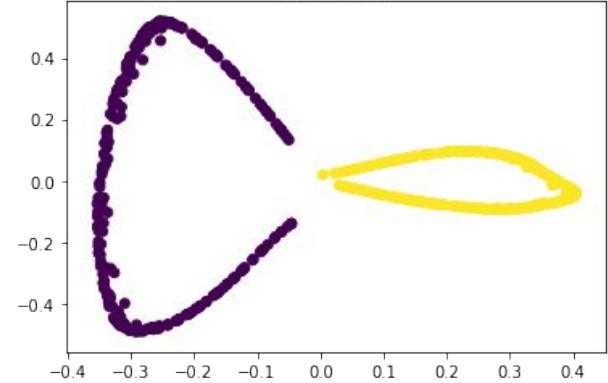
Original Data



PCA



Kernel PCA



Limitations:

1. In PCA we had to eigendecompose a 2×2 matrix. Here we have to do so for a **500×500 matrix**, for 500 data-points. Therefore kPCA is more expensive.
2. To perform the projection we need compute the kernel $K(x, x^{(i)})$ for which we need to **remember all the actual data points**. Memory intensive.

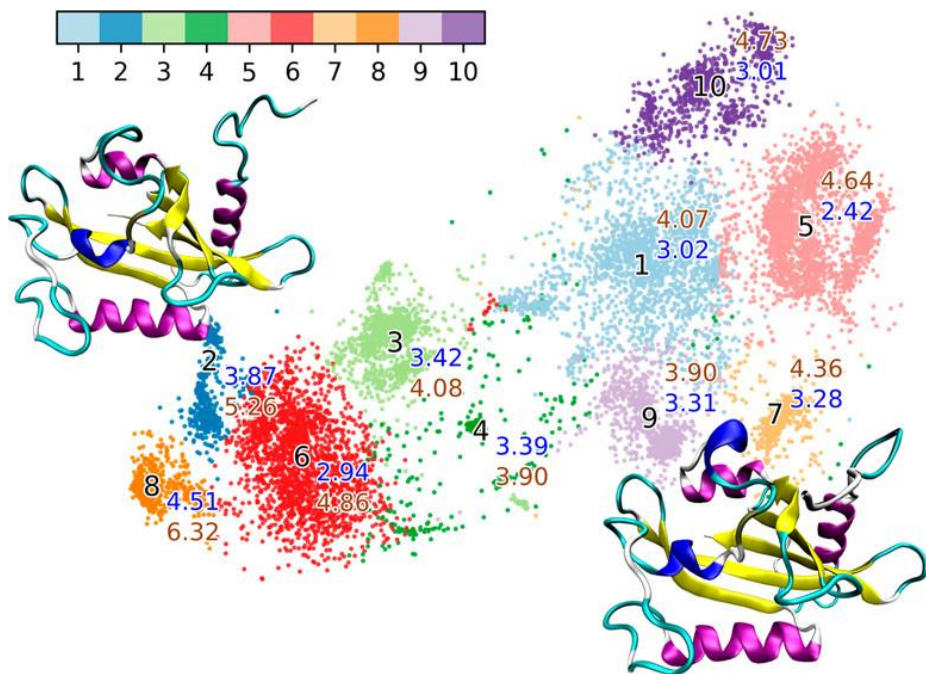
t-distributed Stochastic Neighbor Embedding (t-SNE)

t-SNE learns to embed data points into a lower-dimensional space such that the pairwise distances in the original space are preserved. (Maaten and Hinton, JMLR 2018)

Unlike PCA, it does not involve a projection matrix, so we cannot apply t-SNE learned on a training dataset to project new data points.

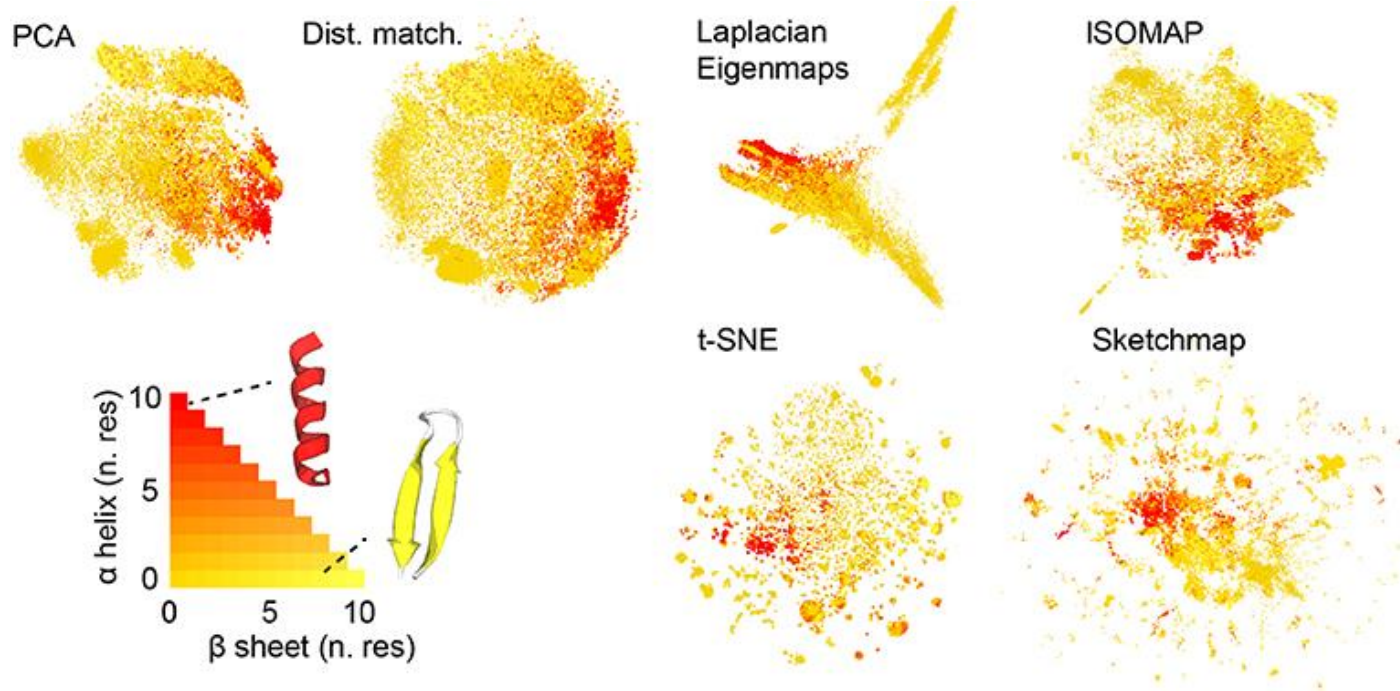
Other important non-linear dimensionality reduction techniques:

- UMAP
- Autoencoder (Neural Network based).



t-SNE on protein structures:
Zhou et al. JCTC 2018

Dimensionality reduction methods on protein folding:



Tribello, Gareth A., and Piero Gasparotto. "Using dimensionality reduction to analyze protein trajectories." *Frontiers in molecular biosciences* 6 (2019): 46.